Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 258 (2025) 1793–1803

Procedia Computer Science

www.elsevier.com/locate/procedia

International Conference on Machine Learning and Data Engineering

AnoLSTM-A Deep Learning Approach for Test Cases Prioritization

Tapas Kumar Choudhury[a], Mousumi Behera[b] , Sanjit Kumar Dash*, Subhendu Kumar Pani[c], Jibitesh Mishra[d]

Biju Patnaik University of Technology, Rourkela, Odisha, India[a,c]
Odisha University of Technology and Research, Bhubaneswar, Odisha, India[b,,d]*

Abstract

In a continuous integration environment, code is often merged into the mainline codebase. As a result, regression testing is essential for ensuring that modifications to the primary codebase do not adversely affect the performance of the software's current characteristics. Test case prioritization (TCP) is a regression-based approach for reordering test cases in a test suite to enhance fault detection performance and shorten test case execution time. In this study, we put forward the AnoLSTM-TCP (Anomaly-Based Long Short-Term Memory Test Case Prioritisation) method to prioritize test cases. Our strategy leverages historical test case data, including features such as duration, execution of the test cases, and list of previous test results to automatically learn patterns and prioritize test cases effectively. Our proposed method goal is to increase the rate at which faults are discovered in test cases. For our experiment, we use an industrial dataset from ABB Robotics Norway (IOFROL and Paint Control). The evaluation metric we used to analyze the effectiveness of our model is APFD values. We compare our novel work with previous works of Random, ROCKET, RETECS, Deepgini, DeepOrder, and HyLSTMTCP. Finally, our experimental results show that AnoLSTM-TCP significantly achieves higher APFD values of 0.85, and 0.90 on IOFROL, and Paint Control datasets which are higher than existing approaches.

Keywords: Continuous Integration, Regression Testing, Test Case Prioritization, LSTM, Deep Learning

1. Introduction

Software testing involves executing a program or system to identify flaws and ensure it meets its intended functionality. It encompasses assessing a program's or system's capabilities to determine if it delivers the desired results. Flaws are common in software modules of moderate size, attributed not to programmer negligence but to the

* Corresponding author. Tel.: 9437990892.
E-mail address: sanjitkumar303@gmail.com

10.1016/j.procs.2025.04.431

inherent complexity of software, where the human ability to manage complexity is limited. Additionally, design faults are inevitable in complex systems [17].

Continuous Integration (CI) simplifies software development by allowing developers to continuously integrate new or updated code into the codebase, while testers provide testing input within time constraints [21]. The major purpose of CI is to shorten the time between code commits and problem fixes, with regression testing playing an important part in assuring software quality [21]. However, regression testing is lengthy, with large test suites taking weeks to complete [18]. As a result, substantial research has been conducted to improve defect identification in CI and reduce regression test execution times [18]. Test case prioritization (TCP), test case minimization (TCM), and test case selection (TCS) are all recognized tactics in CI systems, with test case prioritization acting as an early optimization process [19].TCP is a technique to ordering test cases based on specified criteria to increase fault detection rates, discover regression errors caused by modifications, improve system reliability, detect high-risk defects early, and improve code coverage [20]. Several prioritization strategies, including coverage-based [22], cost-aware [23], and time-aware prioritization [24], have been proposed to address these goals. TCP also helps to mitigate limitations of test dependencies [45] and fault localization [46] by running independent and critical tests first, providing early feedback and reducing the likelihood of cascading failures.

In the last several years, machine learning techniques have achieved popularity in the field of TCP. For this goal, researchers investigated a variety of supervised, unsupervised, and reinforcement learning techniques, such as support vector machine (SVM), k-nearest neighbor (KNN), Random Forest (RF), Logistic Regression, and Linear Discriminant Analysis. Deep learning, a subclass of machine learning, has advanced numerous fields including speech recognition, visual object recognition, and genomics, by enabling computational models with multiple layers to learn complex representations of data [39]. These models leverage backpropagation algorithms to adjust internal parameters and compute representations at each layer based on representations from preceding layers. Inspired by the advancements in deep learning, efforts have been made to implement test case prioritization using deep learning techniques, particularly employing a Long Short-Term Memory (LSTM) based model.

2. In this paper, we introduce AnoLSTM-TCP, a method for TCP in CI environments, leveraging LSTM-based deep learning techniques enriched with anomaly scores as feature inputs. By processing past execution results, our approach aims to learn and exploit temporal correlations within test case histories, discerning subtle deviations indicative of failure-prone areas in the software system across various cycles. Through this iterative learning process, AnoLSTM-TCP maximizes the likelihood of identifying errors in the beginning of the testing procedure. On Comparing to existing methods like Random, ROCKET, RETECS, DeepGini, DeepOrder, LSTMTCP, HyLSTMTCP our approach demonstrates superior fault detection rates, highlighting its effectiveness in enhancing software testing practices. AnoLSTM-TCP's adaptability ensures robust performance across diverse testing environments, offering a scalable solution seamlessly integrated into the CI cycle.

The remaining portions of the paper are structured in the following manner: Section 2 introduces related research on test case prioritization; Section 3 defines the proposed approach's framework; Section 4 describes the evaluation metrics; Section 5 provides details regarding the experimental setup; Section 6 gives results and discussion; and Section 7 concludes the study and suggests future work.

3. Related Work

Test case prioritization for regression testing has received significant interest as a dynamic subject of study in research [8]. Existing efforts on test case prioritization can be divided into three categories: non-machine learning, machine learning, and deep learning-based.

2.1 Non- Machine Learning Based

The suggested approaches within this category commonly rely on heuristic techniques informed by factors such as code coverage [18], [20], [44], predictive models [28], history-based [9] and requirements-based [13], cost-aware [23], and time-aware [18, 34]. A survey was also conducted on these methods in [43].The primary limitation of these techniques, particularly within a CI context is their lack of adaptability to rapidly evolving environments. A substantial portion of prior research emphasizes leveraging code coverage data and analyzing code alterations to prioritize test cases[18]. In [20] several code coverage prioritization techniques are proposed as additional statement coverage, Additional fault-exposing-potential, total fault-exposing-potential, and total statement coverage. Many researchers have used genetic algorithms for TCP [29], [42], [37] but the limitations of using this prioritization technique are limited search efficiency, difficulty in handling high dimensional data, and limited generalization. Carlson et al. (2011)

[32] present a method for TCP using a clustering approach that is based on features like code complexity, code coverage, and historical data. Wang et al. (2017) introduced a method for TCP that is based on the quality-aware technique [30]. This technique will leverage the code-based TCP. Miranda et al. (2018) suggested a TCP approach called FAST [31]. This technology provides scalable similarity-driven TCP by combining white-box and black-box methodologies. Dhiman et al. (2019) introduced a novel TCP technique that depends on the Ant Colony Optimisation (ACO) algorithm [1]. However, this technology has several drawbacks, such as limited adaptability to dynamic environments and difficulty dealing with high-dimensional spaces. Khatibsyarbini et al. (2019) suggested a TCP implementation relies on the Firefly Algorithm [2]. They conducted their experiment on three benchmark programs using suites of tests from the Software-artifact Architecture Repository. Singhal et al. (2022) introduced novel TCS and TCP methodologies utilizing African Buffalo Optimization techniques [4]. Their prioritization approach relies on fault-coverage criteria. Another approach proposed by Marijan et al. (2013) for TCP in a CI environment using the ROCKET case study [11]. ROCKET's prioritization technique relies on historical data failure, test execution duration; and heuristics informed by domain knowledge, with the priority order of the test cases determined using a weighted approach.

2.2 Machine Learning Based

Machine learning (ML) techniques have gained traction in this domain, offering potential advancements through automated decision-making processes. Lachmann et al. (2016) introduced a system-level TCP method leveraging supervised ML and NLP [7]. Their approach integrates black-box meta-data, such as test case execution history and natural language descriptions, for prioritization. However, its applicability is restricted to white-box testing scenarios requiring access to source code. Lachman et al. (2018) presented a black box TCP method employingML algorithm [6], including NN, KNN, SVM, and LR where logistic regression yielded better performance among the models investigated. Their study emphasizes historical and combinatorial learning approaches. Spieker et al. (2017) introduced RETECS, an RL-based strategy for TCS and TCP in a CI environment [12]. Their approach prioritizes based on the duration of test execution, past execution, and failure occurrences. While agents based on ANN were found to perform best, limitations include potential challenges in scaling up to larger networks and the complexity associated with integrating deep learning techniques into the model. Lima et al. (2020) introduced COLEMAN as a solution to the TCPCI problem, employing a Multi-Armed Bandit (MAB) strategy [14]. It employs combinatorial and volatile properties to adaptively generate a suitable Prioritized set of tests for each CI commit based on test case failure data. Bertolino et al. (2020) suggested an approach for TCP that employs two broad ML-driven prioritization strategies: learning-to-rank (LTR) and ranking-to-learn (RTL) [10]. This requires analyzing the code being tested and evaluating the impact of the testing approach on algorithms. The authors propose a class-based dependent network as well as a four-class classification method. Bagherzadeh et al. (2021) investigated TCP in CI environments as a RL problem [5]. They conducted a thorough evaluation across eight topic systems, combining ten RL algorithms with three ordering models: listwise, pointwise, and pairwise. By amalgamating the pairwise ranking model with the ACER algorithms, they attained the highest ranking accuracy. When ample historical and code data are present, an actor-critic RL algorithm such as pairwise-ACER can furnish a stable and versatile solution. Da Roza et al. (2022) proposed a method for TCP in a CI environment (TCPCI) using two ML regression techniques an RF Algorithm and an LSTM deep learning network [15]. Their prioritization technique TCPCI is based on the failure history of test cases. Liu et al. (2023) proposed a technique for prioritizing test cases using self-attention RL in a CI environment [3]. The prioritization method relies on factors including historical outcomes and time intervals. They conducted their experiment on 15 reward functions extracted from 14 different datasets.

2.3 Deep Learning Based

The emergence of deep learning has revolutionized test case prioritization by offering powerful tools for automating and optimizing prioritization strategies. Lousada et al. (2020) introduced NNE-TCP, a revolutionary ML framework for TCP [35]. It identifies updated files during test status shifts and prioritizes based on historical data, utilizing neural network embeddings to identify relevant test instances.Xiao et al. (2020) proposed LSTM-based DL techniques for TCP [27]. Their LSTM-TCP prioritizes based on test case execution history in each cycle. Additionally, they introduced HyLSTMTCP, a hybrid approach combining adaptive and deterministic methods. Shi et al. (2021) performed an empirical investigation on TCP metrics for DNN [16]. They examined 11 TCP metrics, categorizing them into confidence dispersion, mutation uncertainty, surprise adequacy, and mutation rate. Metrics related to surprise adequacy and confidence dispersion showed better performance, while mutation-based metrics were most

effective for detecting all faults. Sharif et al. (2021) Presented a technique for prioritizing test cases utilizing a Deep Neural Network (DNN) named DeepOrder[26]. Test cases were prioritized by DeepOrder considering their duration and execution status. DeepOrderanalyzes effectiveness in terms of fault detection and execution time. Prioritization of test cases is carried out using ROCKET's deterministic method [11]. Li et al. (2023) introduced DeepRank, a method for TCP utilizing DNN [36]. DeepRank ranks test cases based on cross-entropy loss, demonstrating effectiveness in assessing test case prioritization impact and enhancing DNN resilience through retraining. Feng et al. (2020) Introduced DeepGini, a test case prioritization method leveraging the statistical perspective of DNN [34]. DeepGini aims to reduce misclassification chances by assessing the impurity within the test set, facilitating the rapid identification of tests prone to misclassification. However, this model tends to detect fewer faults compared to other methods.

Many approaches are proposed for TCP which often rely on heuristic rules or machine learning-based models. By employing LSTM-based models, our approach AnoLSTM-TCP aims to learn and exploit the temporal correlations present in the test case runtime history. Specifically, the model reviews past test case execution outcomes to identify recurring patterns that indicate potential failure-prone areas within the software system across various testing iterations. By iteratively examining historical data, the LSTM model adapts its approach to test case prioritization, aiming to identify faults early in the testing phase.

4. Framework

In this section, we will describe how the LSTM-based deep learning model prioritizes test cases and its application within the AnoLSTM-TCP approach in a CI environment, specifically within test suites.

3.1 Proposed Framework for AnoLSTM-TCP

Here, we describe our TCP approach using the LSTM model, which is adaptive and well-suited for CI software testing. This approach enables dynamic adjustment of test case priorities in response to evolving requirements over time, accommodating any number of test scenarios.

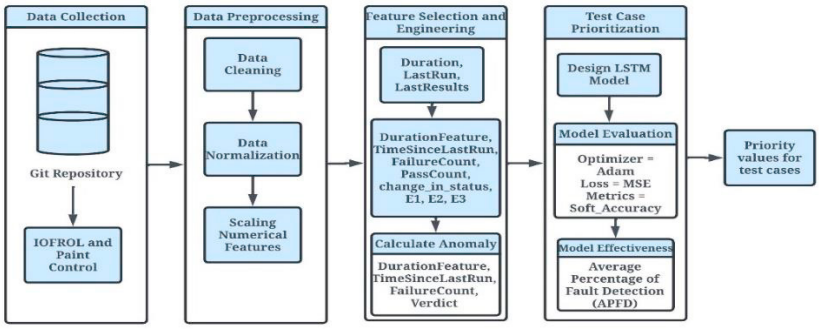

Figure 1: Suggested Framework for Prioritizing Test Cases in CI

The model is trained with the execution history of test cases to predict failure probability, utilizing verdicts indicating whether a test case passed (0) or failed (1). Following failure prediction, test cases are arranged in descending order based on their failure probabilities, allowing for more effective regression testing scheduling in succeeding cycles. This methodology aligns with the goals of AnoLSTM-TCP, which integrates anomaly detection with LSTM models for TCP. The proposed framework for TCP in CI is illustrated in Figure 1. By leveraging unsupervised anomaly detection techniques and recurrent neural networks, AnoLSTM-TCP enhances traditional TCP approaches. Anomaly scores from the isolation forest algorithm serve as input features for the LSTM model, effectively capturing temporal dependencies in test case data. Prediction values from the LSTM model are employed to rank test cases by their potential for critical issues or failures. This enables more efficient resource allocation and risk mitigation in the testing process.

3.2 Data Collection

We collect datasets from https://bitbucket.org/HelgeS/retecs/src/master/DATA/ for our experimentation. Table 1 presents a summary of the datasets, comprising two industrial datasets from ABB Robotics Norway [25] (Paint Control and IOF/ROL). These datasets include Id, Name, Duration, LastRun, LastResults, Verdict, and Cycle features. The ID serves as a distinct numerical label for test execution, while the Name serves as a distinct numerical label for test cases.

Duration approximates the runtime of test cases, LastRun indicates the most recent execution timestamp of test cases, LastResults provides a list of past test results marked as passed (0) or failed (1), Verdict represents the test execution outcome as passed (0) or failed (1), and Cycle indicates the CI cycle number corresponding to the test execution. IOF/ROL comprises 2086 test cases, 320 CI Cycles, and 30,319 Verdicts, with a failure rate of 28.43%. PaintControl encompasses 114 test cases, 352 CI Cycles, and 25,594 Verdicts, with a failure rate of 19.36%. Table 2 details the dataset structure.

Table 1. Summary of the Dataset

Dataset	Test Cases	CI Cycles	Verdicts	Failed
ABB IOF/ROL	2086	320	30,319	28.43%
ABB Paint Control	114	352	25,594	19.36%

Table 2. Structure of the Dataset

Id	Name	Duration	LastRun	LastResults	Verdict	Cycle
1	78082	57262	13-02-2015 16:13:00	[0]	1	1
2	78074	54015	13-02-2015 16:17:00	[1]	1	1
3	78075	61690	13-02-2015 16:19:00	[1,1]	0	1
4	78078	58661	23-02-2015 16:13:00	[0,1,1]	1	3
5	78084	56861	23-02-2015 16:32:00	[1,0,1,1]	0	4

3.3 Data Pre-processing

In the pre-processing phase, we pre-processed our dataset and normalized some of the features such as Duration and LastRun. For the Duration feature, we applied min-max normalization. To normalize the LastRun feature, we initially converted it to a datetime format. Then, computed the time difference between test cases and proceeded to normalize the LastRun feature using min-max normalization. The normalized data was stored in two separate columns, namely DurationFeature and TimeSinceLastRun.

3.4 Feature Selection and Engineering

Features such as Duration, LastRun, and LastResults were selected, and feature engineering focused on extracting features including FailureCount, PassCount, Change_in_status, E1, E2, and E3 from LastResults. FailureCount tallies occurrences of failed executions (1 indicating failure, 0 indicating success), while PassCount does the same for successful executions. Change_in_status tracks transitions between pass (0) and fail (1) statuses in LastResults. E1, E2, and E3 represent sequential executions. Anomalies in the test cases were then calculated using an unsupervised technique, the isolation forest algorithm. The anomaly scores were then used as input features for an LSTM model, which prioritised test cases based on the model's prediction values.

3.5 LSTM Model Design

After extensive experimentation, we developed our proposed LSTM architecture, which consistently demonstrated superior performance across diverse datasets, capturing complex temporal dependencies effectively. The input layer comprises features extracted from test cases, including Duration, LastRun, FailureCount, PassCount, Change_in_status, E1, E2, E3, and Anomaly_Score. These features provide insights into the historical behaviour and anomalies in test case executions. The output layer generates predictions, in the case of TCP, Generating priority scores for test cases by processing the input data through the LSTM layers. Through these approaches, our LSTM model effectively prioritizes test cases, offering insights into critical issues and facilitating efficient resource allocation in software testing processes. The algorithm for the LSTM-based TCP model is summarized in Table 3

Table 3: Algorithm for LSTM-based Test Case Prioritization (TCP) Model

Algorithm: Designing an LSTM Model for Test Case Prioritization
Input: Features extracted from the test cases
Output: Priority Values for the test cases
Step 1: Start **Step 2:**Import necessary library packages **Step 3:** Load the dataset containing test cases **Step 4:**Perform data pre-processing and Feature engineering **Step 5:**Split the dataset into training and testing sets with an 80:20 ratio. **Step 6:** Apply the LSTM model$h_t = O_t * \tanh(C^t)$

Step 6.1 Define a Sequential model
Step 6.2 Add three LSTM layers
Step 6.2.1 Initialize in each layer with,Units = 50, and Activation Function = ReLU
Step 6.3 Introduce regularization after each layer, Dropout = 0.2
Step 6.4 Add three Dense Layer
Step 6.4.1 Initialize, 1st layer with Units = 32 and 2nd layer with Units = 16, Final Output Layer with Units = 1 and Activation Function = ReLU
Step 7: LSTM Model Compilation
Step 7.1 Optimizer = Adam, Learning_rate = 0.001, Loss = Mean Squared Error, and Metric = Soft_accuracy
Step 8: Training
Step 8.1 Initialize Early Stopping
Step 8.1.1 Monitor = Val_Loss, Patience = 50, Restore_best_weights = True
Step 8.2 Initialize Epoch
Step 8.1.2 Epoch = 500, Batch_Size = 32
Step 9: Plot Loss and Accuracy Graph
Step 10: Evaluate model performance using error metrics MSE, RMSE, and R2_Score.
Step 11: Exit

Furthermore, we observed substantial improvements in model performance and generalization by integrating anomaly features. These features detect deviations in test case behaviour, providing insights into various execution patterns. By aiding LSTM models in understanding the complexity and variability of test case behaviours, anomaly features enhance their ability to differentiate between informative and less informative cases. Additionally, they enhance adaptability to dynamic testing environments by capturing emerging anomalies or trends, enabling continuous updates to prioritization strategies. 'Anomaly_Score' emerges as the most influential feature in Figure 2, underlining its pivotal role in prioritization decisions, alongside 'TimeSinceLastRun' and 'FailureCount'.

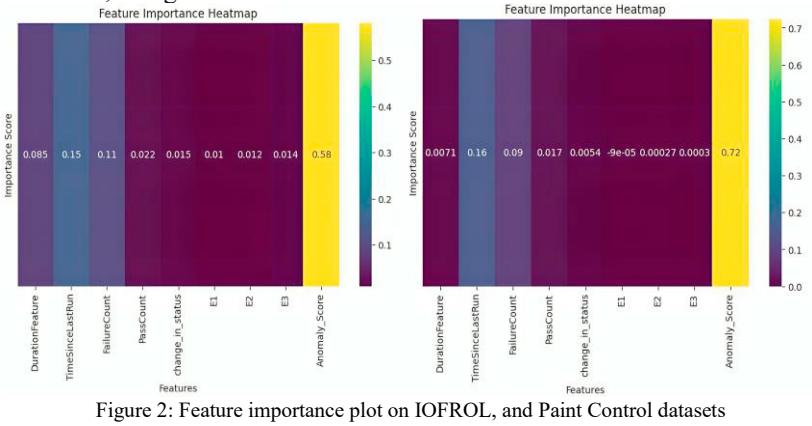

Figure 2: Feature importance plot on IOFROL, and Paint Control datasets

5. Experimental Metrics

This section will cover the metrics employed to assess the proposed model for TCP.

4.1 Average Percentage of Fault Detection (APFD)

The analysis of our experiment was conducted using APFD [41]. This metric tracks the cumulative percentage of defects found as test cases are processed in a specified order within a set of test cases. APFD values vary from 0 to 100, with higher values suggesting quicker fault detection rates. The model's effectiveness is enhanced with increasing APFD values. APFD is determined by

$$\text{APFD} = 1 - \frac{T*F1+T*F2+\cdots+T*Fm}{n*m} + \frac{1}{2*n} \quad (1)$$

Let T be a test suite with n test cases, and F is the set of m faults discovered by T. The definition of TFi, the first test case in the order T0 of T that reveals fault i, applies for each fault i in F.

4.2 Normalized Average Percentage of Faults Detected (NAPFD)

$$\text{NAPFD} = p - \frac{T*F1+T*F2+\cdots+T*Fm}{n*m} + \frac{p}{2*n} \quad (2)$$

It assesses the effectiveness of prioritization by taking into account both identifying defects and detection time. It is given by

Let p be the measure of effectiveness of the prioritized test suite, described as the proportion of faults discovered by the prioritized test suite to the total number of faults identified.

4.3 Mean Squared Error (MSE)

MSE [40]is a statistic used to calculate the performance of regression models. It calculates the mean of the squared errors, which are the differences between the actual and predicted values. It is given by

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2 \quad (3)$$

4.4 R2 Score

A statistical measure known as R-squared measures the amount of variation in the dependent variable that can be predicted by the independent variables in a regression model. It is often interpreted as the proportion of variation that the model explains and ranges from 0 to 1.

4.5 Root Mean Squared Error(RMSE)

The average magnitude of the errors between expected and actual values is determined by the RMSE, taking into account both the magnitude and direction of the errors. It is determined as the square root of the average squared deviations between anticipated and obtained values. The expression is as follows

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2} \quad (4)$$

4.6 Time to detect First Fault (TF)

This metric calculates how long it takes a prioritized test suite to identify the first error. Given a prioritized test suite F = {F1, F2, ..., Fn}, a collection of faults discovered by F as P = {P1, P2, ..., Pm}, and a collection of fault detection timings for each Pi as T = {T(P1), T(P2), ..., T(Pm)}, then TF = T(Pi), where T(Pi) < T(Pj), ∀T(Pj) ∈ T. This measure determines the early fault detection capacity.

4.7 Time to detect Last Fault (TL)

This metric calculates how long it takes a prioritized test suite to identify the last error. Given a prioritized test suite F = {F1, F2, ..., Fn}, a collection of faults detected by F as P = {P1, P2, ..., Pm}, and a collection of fault detection timings for each Pi as T = {T(P1), T(P2), ..., T(Pm)}, then TL = T(Pi), where T(Pi) > T(Pj), ∀T(Pj) ∈ T. This measure also determines the early fault detection capacity.

4.8 Time to detect Average Fault (TA)

This measure determines the average time taken by a prioritized test suite to detect errors. Given a prioritized test suite F = {F1, F2, ..., Fn}, a collection of faults detected by F as P = {P1, P2, ..., Pm}, and a collection of fault detection timings for each Pi as T = {T(P1), T(P2), ..., T(Pm)}, then the mean of all fault detection times in T is then used to compute the average time to detect faults (TA). It provides an overall assessment of the efficiency of fault detection within the prioritized test suite.

4.9 Time to Prioritize Test Cases (PT)

This metric assesses the total time taken for execution of a prioritization algorithm, denoted as ranking time (PT).

6. Experimental Setup

We completed our data analysis and LSTM-based model construction with Jupyter Notebook, an interactive work environment, and Anaconda [38], a distribution platform that includes a full collection of Data Science tools and packages. Implementation of the training and prediction steps for the LSTM model occurs in Python using TFTS (TensorFlow Time Series) [33]. To assess the efficacy of our approach, the set of experiments was carried out in Python 3.9.7, Anaconda 23.11.0, and Tensorflow 2.12.0 on a machine configured with an Intel core i3 processor (12th Gen Intel(R) Core (TM) i3-1215U 1.20 GHz), 8.00 GB of RAM, and a 64-bit operating system running Windows 11. To assess the performance of our TCP approach in a CI environment, we formulated specific research questions based on our experimentation.

RQ1: What is the predictive accuracy of the AnoLSTM-TCP and What is the effectiveness of the AnoLSTM-TCP approach in terms of fault detection and time efficiency?

RQ2: How does our experiment outperform existing techniques for test case prioritization?

7. Results and Discussion

TCP optimizes testing efforts, allocating resources efficiently to detect faults early. AnoLSTM-TCP leverages LSTM-based deep learning with anomaly scores, discerning subtle execution history deviations. Our model enhances fault detection rates in CI environments. This section offers an overview of the experiment results and examines their significance in addressing our research questions.

A. Prioritization of test cases using the AnoLSTM-TCP model:

Table 4, provides a snapshot of the top five prioritized test cases utilizing the AnoLSTM-TCP methodology. Each row represents a specific test case, featuring pertinent details including ID, Duration, LastRun, LastResults, verdict, cycle, FailureCount, PassCount, change_in_status, and calculated priority (CalcPrio) score. The priority score, generated by the AnoLSTM-TCP model, indicates the relative significance of each test case within the testing framework. Additionally, the ranking column denotes the position of each test case in the prioritized sequence, with lower rankings indicating higher priority.

Table 4: Prioritized Test Cases Based on the AnoLSTM-TCP Approach

Id	DurationFeature	TimeSinceLastRun	Failure Count	PassCount	Change_in_status	E1	E2	E3	Anomaly_Score	Verdict	CalcPrio	Ranking
16754	0.062220	0.519328	3	0	0	1	1	1	0.198983	1	1.089061	1
16793	0.062220	0.520537	4	0	0	1	1	1	0.209289	1	1.084475	2
14775	0.063984	0.457981	4	0	0	1	1	1	0.224004	1	1.080452	3
16826	0.062220	0.521560	5	0	0	1	1	1	0.229878	1	1.076265	4
12448	0.062597	0.385846	6	9	5	0	1	1	0.296867	1	1.075240	5

RQ1: Here, we will discuss the predictive accuracy of our proposed model for test case prioritization. First, we assess our model's performance by measuring the mean squared error. Table 5 represents our proposed model mean square error values for the two datasets. Lower MSE values show the improved performance of the model. The proposed model achieves a high R-squared value, indicating a strong fit of the model to the dataset.

Table 5: Model Evaluation Results on the two datasets

Datasets	MSE	R2	RMSE	APFD	NAPFD	TF(s)	TL(s)	TA(s)	PT(s)
IOF/ROL	0.033	0.83	0.183	0.85	0.59	0.020791	1.651858	0.836324	6.01
Paint Control	0.001	0.99	0.039	0.90	0.63	0.025453	1.49251	0.758982	4.84

In Figure 3 Illustrates the loss function for the IOFROL and Paint Control datasets. The loss function serves as a measure of the disparity between the predicted values generated by our model and the actual ground truth values within our dataset. By visualizing the loss graph, we can gain insights into how effectively our model learns from the data and minimizes errors during the training process.

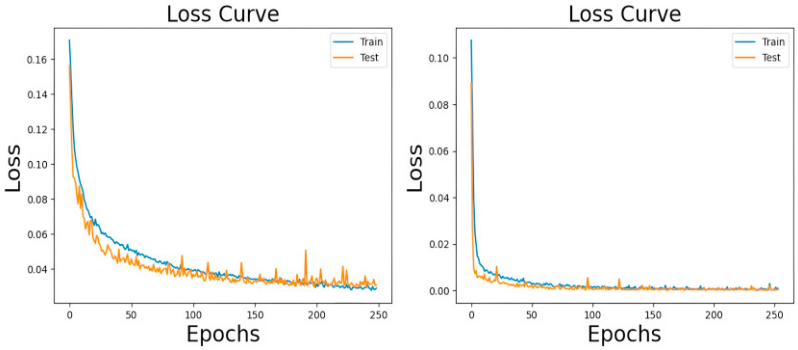

Figure 3: Loss graph on IOFROL, and Paint Control

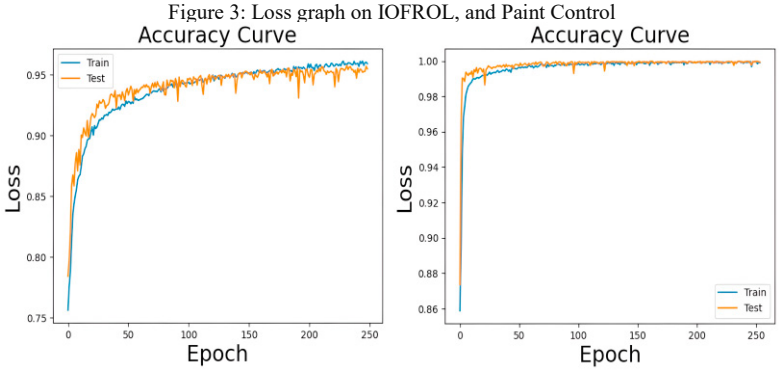

Figure 4: Accuracy graph on IOFROL, and Paint Control

In Figure 4, we represent the accuracy curve on the IOFROL, and Paint Control datasets. The accuracy curve shows, in graphical form, how successfully our model generalizes to new data across multiple training epochs or iterations. We can understand the convergence and stability of our model's predictions by looking at the patterns and variations in accuracy over time. Let's take a deeper look at the accuracy curve to evaluate our model's performance.
Figure 5, represents the NAPFD values for different CI cycles across datasets. These visualizations provide insights into how the performance metrics NAPFD evolve across different CI cycles and datasets, offering a comparative analysis of fault detection effectiveness in software testing scenarios. For the IOFROL dataset, NAPFD values fluctuate across different CI cycles, with the highest value observed at 200 cycles and the lowest at 10 cycles. In the case Paint Control dataset, NAPFD values vary significantly with CI cycles, peaking at 10 cycles and dropping notably at 100 cycles before showing some recovery at 200 cycles.

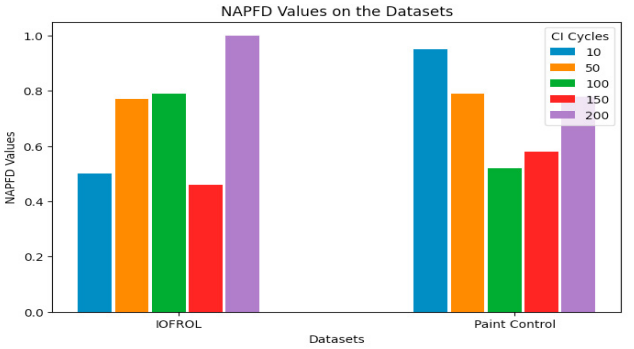

Figure 5: NAPFD values on different CI Cycles

RQ2: In this, we will compare the existing approach for TCP with the proposed approach. Existing approaches like Random, ROCKET [11], RETECS-N [12], DeepGini [34], DeepOrder [26], LSTMTCP [27], HyLSTMTCP [27]. In Figure 6, We have conducted a comparative evaluation of the proposed model against the existing model. It is a comparative analysis of the APFD value achieved by seven different models on two datasets IOFROL, and Paint Control.In our comparative analysis of test case prioritization models, the AnoLSTM-TCP model emerges as the most effective approach among the considered techniques. For instance, when compared to the Random model, which selects test cases randomly resulting in limited prioritization efficacy with APFD values of 0.51 and 0.50 for the 'IOFROL' and 'Paint Control' datasets respectively, AnoLSTM-TCP showcased significantly superior performance with APFD values of 0.85 and 0.90 across the same datasets. Additionally, AnoLSTM-TCP surpassed other techniques like ROCKET, RETECS-N, and deep learning-based approaches such as DeepGini and DeepOrder. Even within the LSTM-based models' category, AnoLSTM-TCP outperformed its counterparts, including LSTMTCP and HYLSTMTCP. These results highlight AnoLSTM-TCP as a leading approach in TCP, offering significant improvements in fault detection capabilities and potential enhancements to software testing processes.

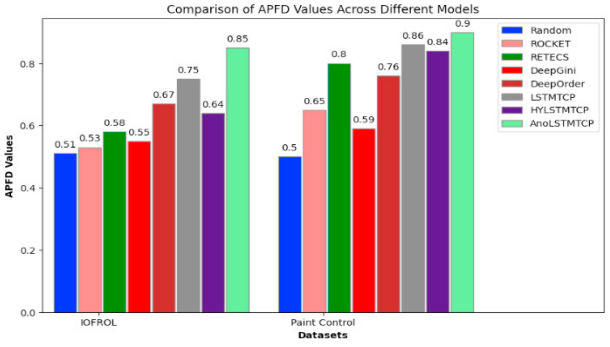

Figure 6: Comparing the APFD values across the datasets

In assessing the time effectiveness of AnoLSTM-TCP compared to DeepOrder and DeepGini, it's evident that AnoLSTM-TCP outperforms both models across TF, TL, and TA metrics, as shown in Figure 7. Specifically, AnoLSTM-TCP demonstrates superior efficiency in detecting faults early in the testing process, as indicated by its significantly lower time to detect first fault (TF) and time to detect last fault (TL) values compared to DeepOrder and DeepGini. Furthermore, AnoLSTM-TCP exhibits a more efficient overall average time (TA), highlighting its ability

to swiftly identify faults on average across the testing datasets. These findings underscore the efficacy of AnoLSTM-TCP in accelerating fault detection processes, leading to faster identification and resolution of critical issues in software systems. This superiority in fault detection effectiveness positions AnoLSTM-TCP as a more efficient and reliable solution for optimizing testing workflows compared to DeepOrder and DeepGini.

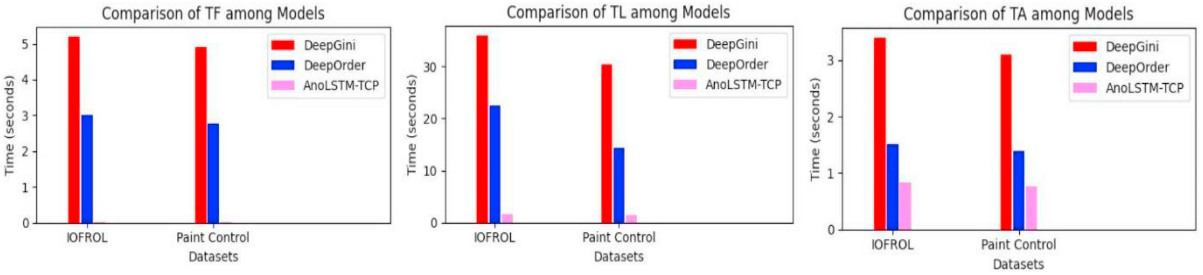

Figure 7: Comparison of Time Effectiveness Metrics between DeepGini, DeepOrder, and AnoLSTM-TCP Models

8. Conclusion and Future Work

The AnoLSTM-TCP approach offers a promising solution for software test case prioritization by leveraging anomaly features and LSTM models. Through experimentation, we've shown the effectiveness of incorporating anomaly features in LSTM models, improving prioritization performance. By capturing deviations in test case behavior, AnoLSTM-TCP provides valuable insights into execution complexity, enabling more informed prioritization decisions. Our analysis highlights the critical role of anomaly scores, temporal aspects, and test case history in guiding prioritization strategies. The AnoLSTM-TCP approach presents numerous opportunities for future enhancement in software testing. Integrating additional features tailored to specific domains can enrich the model's understanding of test case behaviors, enhancing prioritization accuracy. Fine-tuning hyper parameters and anomaly detection algorithms offers avenues for optimizing performance and adaptability. Exploring ensemble techniques, including combining multiple LSTM models or integrating other machine learning algorithms, can enhance robustness and generalization. Real-world deployment of AnoLSTM-TCP promises practical utility and scalability, while dynamic adaptation of prioritization strategies to evolving testing environments can lead to more agile practices. Improving interpretability and explain ability is crucial for stakeholders' trust and adoption. Through these explorations, AnoLSTM-TCP holds potential for advancing software testing efficiency and effectiveness.

References

[1] Dhiman, R., & Chopra, V. (2019, March). Novel approach for test case prioritization using ACO algorithm. In *2019 IEEE 2nd International Conference on Information and Computer Technologies (ICICT)* (Pp. 292-295). IEEE.

[2] Khatibsyarbini, M., Isa, M. A., Jawawi, D. N., Hamed, H. N. A., & Suffian, M. D. M. (2019). Test case prioritization using firefly algorithm for software testing. *IEEE access*, 7, 132360-132373.

[3] Liu, B., Li, Z., Zhao, R., & Shang, Y. (2023, June). A Self-attention Agent of Reinforcement Learning in Continuous Integration Testing. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)* (Pp. 886-891). IEEE.

[4] Singhal, S., Jatana, N., Subahi, A. F., Gupta, C., Khalaf, O. I., & Alotaibi, Y. (2023). Fault Coverage-Based Test Case Prioritization and Selection Using African Buffalo Optimization. *Computers, Materials & Continua*, 74(3).

[5] Bagherzadeh, M., Kahani, N., & Briand, L. (2021). Reinforcement learning for test case prioritization. *IEEE Transactions on Software Engineering*, 48(8), 2836-2856.

[6] Lachmann, R. (2018, June). Machine learning-driven test case prioritization approaches for black-box software testing. In *The European test and telemetry conference,* Nuremberg, Germany.

[7] Lachmann, R., Schulze, S., Nieke, M., Seidl, C., & Schaefer, I. (2016, December). System-level test case prioritization using machine learning. In *2016 15th IEEE International Conference on Machine Learning and Applications (ICMLA)* (pp. 361-368). IEEE.

[8] Khatibsyarbini, M., Isa, M. A., Jawawi, D. N., & Tumeng, R. (2018). Test case prioritization approaches in regression testing: A systematic literature review. *Information and Software Technology*, 93, 74-93.

[9] Kim, J. M., & Porter, A. (2002, May). A history-based test prioritization technique for regression testing in resource constrained environments. In *Proceedings of the 24th international conference on software engineering* (pp. 119-129).

[10] Bertolino, A., Guerriero, A., Miranda, B., Pietrantuono, R., & Russo, S. (2020, June). Learning-to-rank vs ranking-to-learn: Strategies for regression testing in continuous integration. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (pp. 1-12).

[11] Marijan, D., Gotlieb, A., & Sen, S. (2013, September). Test case prioritization for continuous regression testing: An industrial case study. In *2013 IEEE International Conference on Software Maintenance* (pp. 540-543). IEEE.

[12] Spieker, H., Gotlieb, A., Marijan, D., & Mossige, M. (2017, July). Reinforcement learning for automatic test case prioritization and selection in continuous integration. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis* (pp. 12-22).

[13] H. Srikanth and L. Williams, "On the economics of requirement based test case prioritization," *ACM SIGSOFT Softw. Eng. Notes*, vol. 30, no. 4, pp. 1–3, 2005.
[14] Lima, J. A. P., & Vergilio, S. R. (2020). A multi-armed bandit approach for test case prioritization in continuous integration environments. *IEEE Transactions on Software Engineering*, 48(2), 453-465.
[15] Da Roza, E. A., Lima, J. A. P., Silva, R. C., & Vergilio, S. R. (2022, March). Machine learning regression techniques for test case prioritization in a continuous integration environment. In *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (pp. 196-206). IEEE.
[16] Shi, Y., Yin, B., Zheng, Z., & Li, T. (2021, December). An Empirical Study on Test Case Prioritization Metrics for Deep Neural Networks. In *2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)* (pp. 157-166). IEEE.
[17] Pan, J. (1999). Software testing. *Dependable Embedded Systems*, 5(2006), 1.
[18] Rothermel, G., Untch, R. H., Chu, C., & Harrold, M. J. (2001). Prioritizing test cases for regression testing. *IEEE Transactions on software engineering*, 27(10), 929-948.
[19] Yoo, S., & Harman, M. (2012). Regression testing minimization, selection and prioritization: a survey. *Software testing, verification and reliability*, 22(2), 67-120.
[20] Rothermel, G., Untch, R. H., Chu, C., & Harrold, M. J. (1999, August). Test case prioritization: An empirical study. In *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99).*'Software Maintenance for Business Change'(Cat. No. 99CB36360) (pp. 179-188). IEEE.
[21] Duvall, P. M., Matyas, S., & Glover, A. (2007). Continuous integration: improving software quality and reducing risk. Pearson Education.
[22] Rothermel, G., & Harrold, M. J. (1993, September). A safe, efficient algorithm for regression test selection. In *1993 Conference on Software Maintenance* (pp. 358-367). IEEE.
[23] Elbaum, S., Malishevsky, A., & Rothermel, G. (2001, May). Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering. ICSE 2001* (pp. 329-338). IEEE.
[24] Walcott, K. R., Soffa, M. L., Kapfhammer, G. M., & Roos, R. S. (2006, July). Timeaware test suite prioritization. In *Proceedings of the 2006 international symposium on Software testing and analysis* (pp. 1-12).
[25] https://bitbucket.org/HelgeS/retecs/src/master/DATA/
[26] Sharif, A., Marijan, D., & Liaaen, M. (2021, September). DeepOrder: Deep learning for test case prioritization in continuous integration testing. In *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)* (pp. 525-534). IEEE.
[27] Xiao, L., Miao, H., Shi, T., & Hong, Y. (2020). LSTM-based deep learning for spatial–temporal software testing. *Distributed and Parallel Databases*, 38, 687-712.
[28] Tahat, L., Korel, B., Harman, M., & Ural, H. (2012). Regression test suite prioritization using system models. *Software Testing, Verification and Reliability*, 22(7), 481-506.
[29] Malhotra, R., & Bharadwaj, A. (2012). Test case prioritization using genetic algorithm. *International Journal of Computer Science and Informatics*, 2(3), 63-6.
[30] Wang, S., Nam, J., & Tan, L. (2017, August). QTEP: Quality-aware test case prioritization. In *Proceedings of the 2017 11th Joint Meeting on foundations of software engineering* (pp. 523-534).
[31] Miranda, B., Cruciani, E., Verdecchia, R., & Bertolino, A. (2018, May). FAST approaches to scalable similarity-based test case prioritization. In *Proceedings of the 40th International Conference on Software Engineering* (pp. 222-232).
[32] Carlson, R., Do, H., & Denton, A. (2011, September). A clustering approach to improving test case prioritization: An industrial case study. In *2011 27th IEEE International Conference on Software Maintenance (ICSM)* (pp. 382-391). IEEE.
[33] GitHub - tgjeon/TensorFlow-Tutorials-for-Time-Series: TensorFlow Tutorial for Time Series Prediction
[34] Feng, Y., Shi, Q., Gao, X., Wan, J., Fang, C., & Chen, Z. (2020, July). Deepgini: prioritizing massive tests to enhance the robustness of deep neural networks. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis* (pp. 177-188).
[35] Lousada, J., & Ribeiro, M. (2020). Neural network embeddings for test case prioritization. *arXiv preprint arXiv:2012.10154*.
[36] Li, W., Zhang, Z., Jian, Y., Liu, C., & Huang, Z. DeepRank: Test Case Prioritization for Deep Neural Networks.
[37] Kaur, A., & Goyal, S. (2011). A genetic algorithm for regression test case prioritization using code coverage. *International journal on computer science and engineering*, 3(5), 1839-1847.
[38] Rolon-Mérette, D., Ross, M., Rolon-Mérette, T., & Church, K. (2016). Introduction to Anaconda and Python: Installation and setup. Quant. Methods Psychol, 16(5), S3-S11.
[39] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. nature, 521(7553), 436-444.
[40] Das, K., Jiang, J., & Rao, J. N. K. (2004). Mean squared error of empirical predictor.
[41] Mor, A. (2014). Evaluate the effectiveness of test suite prioritization techniques using APFD metric. *IOSR Journal of Computer,* 16(4), 47-51.
[42] Konsaard, P., & Ramingwong, L. (2015, June). Total coverage based regression test case prioritization using genetic algorithm. In *2015 12th international conference on electrical engineering/electronics, computer, telecommunications and information technology (ECTI-CON)* (pp. 1-6). IEEE.
[43] Mohanty, S., Acharya, A. A., & Mohapatra, D. P. (2011). A survey on model based test case prioritization. *International Journal of Computer Science and Information Technologies*, 2(3), 1042-1047.
[44] Zhu, Y., & Liu, F. (2023). Test case prioritization algorithm based on improved code coverage. *IAENG International Journal of Computer Science*, 50(2).
[45] Alakeel, A. M. (2022). Dependency detection and repair in web application tests. *IAENG International Journal of Computer Science*, 49(2).
[46] Saxena, A., Bhatnagar, R., & Srivastava, D. K. (2022). Software fault localization: Techniques, issues and remedies. *IAENG International Journal of Computer Science*, 49(2).